

SMART CHAT AI WITH MEMORY

Project Introduction & Technical Documentation

Smart Chat AI with Memory is a production-ready, AI-powered backend chat application built using modern cloud-native architecture and DevOps best practices. It delivers a scalable REST API capable of powering intelligent conversations with persistent memory, secure authentication, real-time analytics, and high-performance caching.

The system integrates Google Gemini for AI-driven conversation generation, uses PostgreSQL for structured, persistent data storage, and leverages Redis as a high-speed caching layer to reduce latency and API costs. The entire application is containerized with Docker and served through an NGINX reverse proxy, ensuring consistent, secure, and reproducible deployments across environments.

Beyond the core chat functionality, the project implements a full CI/CD pipeline using GitHub Actions — automating build, test, and deployment stages to an AWS EC2 production server, complete with automated database migrations and health verification.

Repo link : - <https://github.com/Akku690/ai-chat.git>

1. Project Introduction

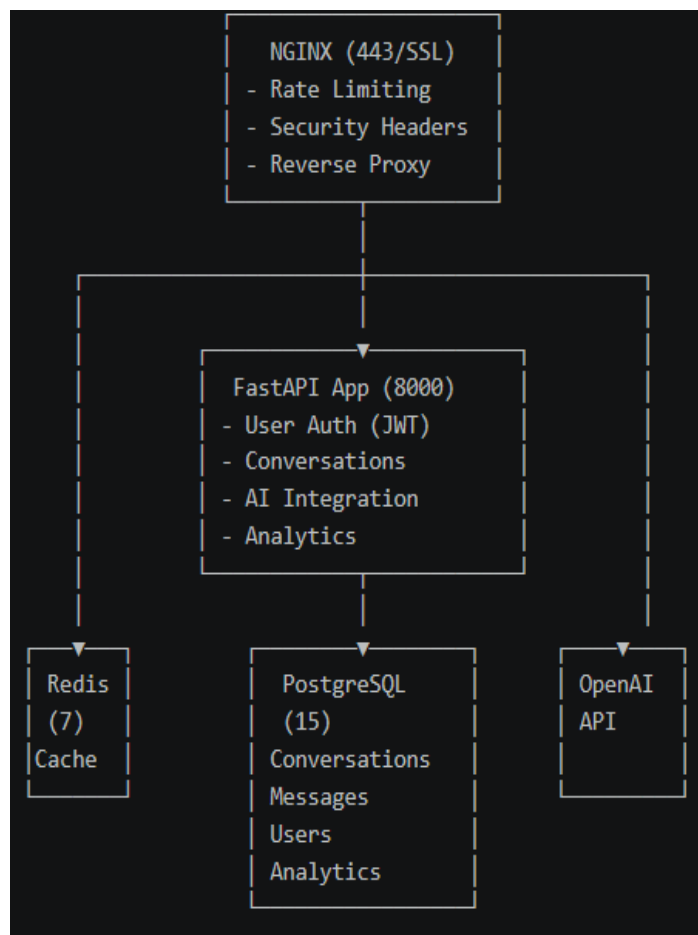
Smart Chat AI with Memory is a production-ready AI-powered backend chat application designed using modern cloud-native architecture and DevOps practices. The project provides a scalable REST API system capable of handling intelligent AI conversations with persistent memory, secure authentication, real-time analytics, and high-performance caching. The application is built primarily using FastAPI, PostgreSQL, Redis, Docker, and NGINX.

Key Features

- User authentication with JWT tokens (stateless, secure)
- Persistent conversation and message storage via PostgreSQL
- Redis caching layer for high-performance response delivery
- AI integration with Google Gemini (and optional OpenAI GPT-3.5)
- Real-time analytics and usage statistics
- Docker containerization for reproducible deployments
- Production-grade security: bcrypt, HTTPS, CORS, rate limiting
- Health monitoring endpoints for all services

2. System Architecture Diagram

The application follows a layered, microservice-inspired architecture where each component has a clearly defined role. All services communicate over an internal Docker network, with NGINX acting as the public-facing entry point.



2.1 Component Summary

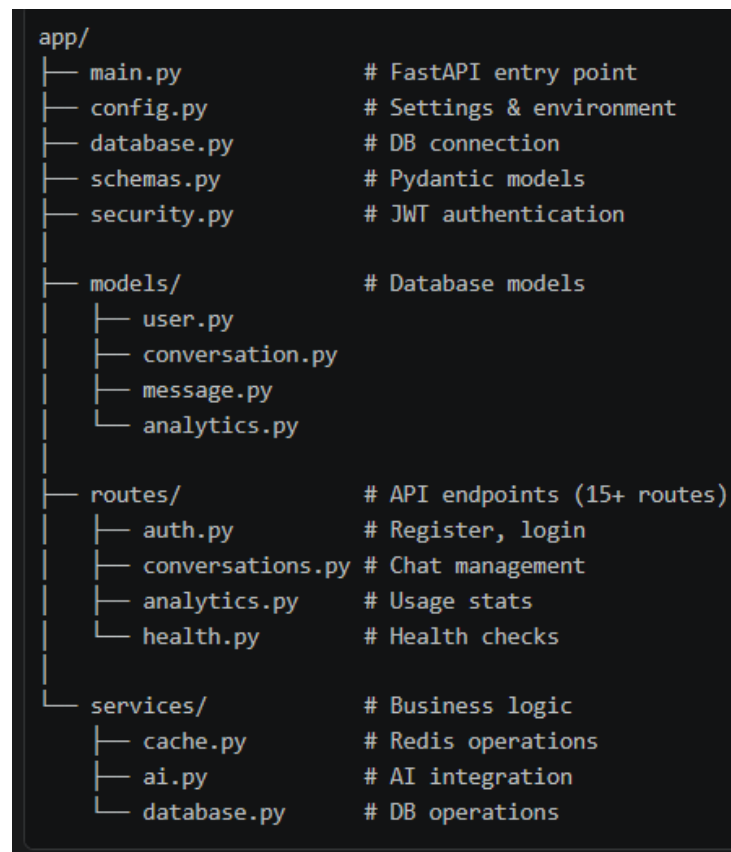
Component	Port	Purpose
FastAPI	8000	Backend API
PostgreSQL	5432	Data Storage
GeminiAI API	HTTPS	Communication and response
Redis	6379	Caching layer
NGINX	80/443	Reverse proxy + SSL

2.2 Hardware & Software Requirements

CPU	2 cores minimum (4 recommended for EC2)
RAM	4 GB minimum (t3.medium has 4 GB)
Storage	5–10 GB free disk space
OS	Ubuntu 22.04 LTS (EC2)
Docker Desktop	v20.10+
Docker Compose	v5.1.3
Python	3.11
Git	Any recent version

3. File Structure

The project is organized into clearly separated layers — routing, services, models, and configuration — following FastAPI best practices for scalable API development.



3.1 Key Files Explained

- **app/main.py** FastAPI application entry point. Registers all routers, configures CORS, and starts the Uvicorn server on port 8000.
- **app/config.py** Centralized settings management using Pydantic's BaseSettings — reads all environment variables from .env file.
- **app/database.py** SQLAlchemy engine, session factory, ORM base class, and the init_db() function for automatic table creation.
- **app/security.py** JWT token creation, validation, and bcrypt password hashing/verification utilities.
- **app/routes/** All API endpoints organized by domain: auth.py, conversations.py, analytics.py, and health.py.
- **app/services/ai.py** AIService class that calls the Gemini API, manages prompts, and handles response caching.
- **app/services/cache.py** RedisCache class — wraps the Redis client with async get/set/delete/pattern-clear operations.

4. API Endpoints Implemented

Authentication		
POST	/auth/register	# Create user account
POST	/auth/login	# Get JWT token
GET	/auth/me	# Get current user
Conversations		
POST	/conversations	# Create conversation
GET	/conversations	# List all conversations
GET	/conversations/{id}	# Get conversation
PUT	/conversations/{id}	# Update title
DELETE	/conversations/{id}	# Delete conversation
Chat & Messages		
GET	/conversations/{id}/messages	# Get chat history
POST	/conversations/{id}/chat	# Send message & get AI response
Analytics		
GET	/analytics/stats	# User usage statistics
Monitoring		
GET	/health	# Service health check

4.1 API Endpoint: Send Message to

AI Endpoint URL :

POST /conversations/{conversation_id}/chat

Request Format :

```
{
  "message": "Hello, tell me a joke"
}
```

Response Format :

```
{
  "conversation_id": 1,
  "user_message": {
    "id": 5,
    "role": "user",
    "content": "Hello, tell me a joke",
    "is_cached": false
  },
  "assistant_message": {
    "id": 6,
    "role": "assistant",
    "content": "Why did the developer go broke? Because he used up all his cache!",
    "is_cached": false
  },
  "tokens_used": 42,
  "response_time_ms": 1250
}
```

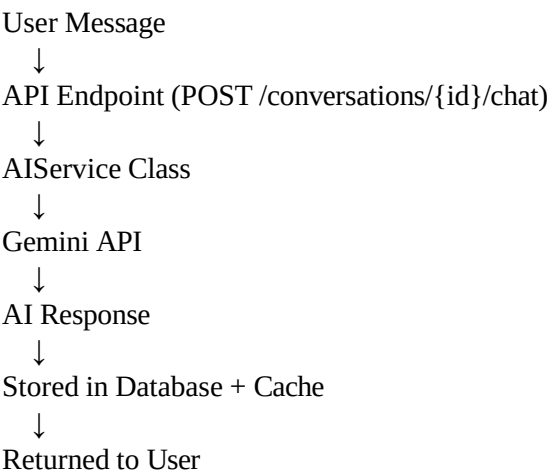
5. Ai Bot Setup

The application uses Google Gemini as its primary AI model. Gemini provides state-of-the-art language understanding and generation capabilities via a secure HTTPS API. The AIService class abstracts all AI communication and adds intelligent caching to minimize API costs.

Current AI Configuration

Property	Details
Primary AI	Google Gemini API
Model Used	gemini-pro
API Key	AIzaSyBVs_Uj5jmeLEqSYB5niEcXi59_gNHTw4M

How AI Service Works



Configuration in .env

```
# AI Configuration (Gemini API)
GEMINI_API_KEY=AIzaSyBVs_Uj5jmeLEqSYB5niEcXi59_gNHTw4M
AI_MODEL=gemini-pro
```

Website url

<https://aistudio.google.com/app/api-keys?project=gen-lang-client-0851811133>

Gemini API Key Setup

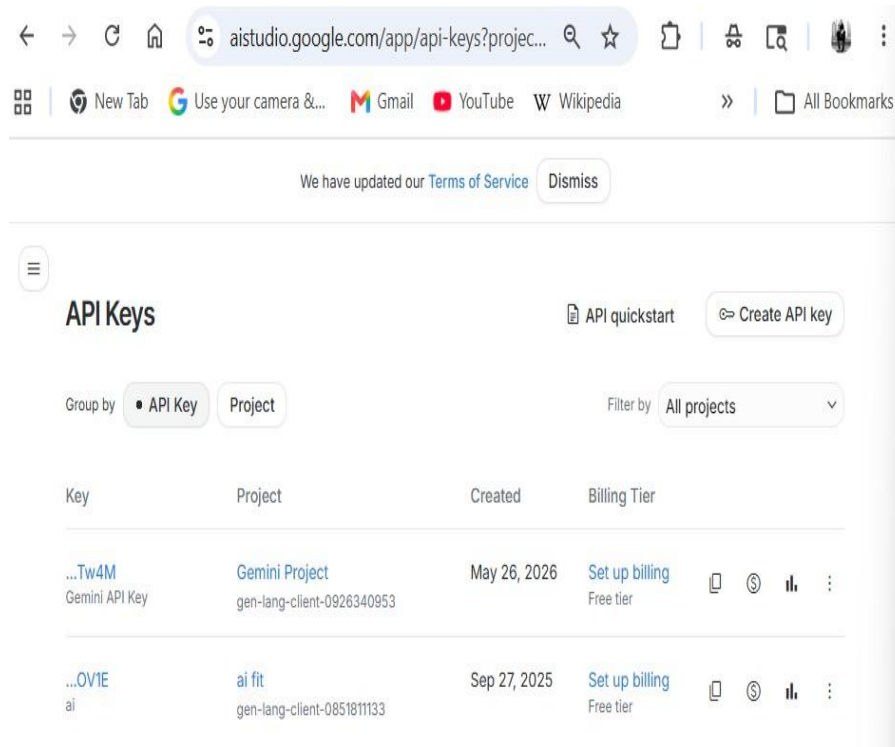


Fig 4 — Google AI Studio: API Key Management Dashboard

The Gemini API key is generated at aistudio.google.com and added to the `.env` file under `GEMINI_API_KEY`. The Free Tier is sufficient for development and testing. Production use should enable billing for higher rate limits.

6.1 App Setup

1. Database

PostgreSQL 15

The application uses PostgreSQL hosted on Render.com . PostgreSQL is the industry-standard relational database. It offers ACID compliance, powerful query capabilities, and excellent scalability — perfect for storing structured chat data.

The application stores the real time data in a database . Data access is managed through the **API keys from a specific website** to interact with **PostgreSQL**.

Code :

```
database.py 3 X
app > database.py > init_db
1 from sqlalchemy import create_engine
2 from sqlalchemy.orm import sessionmaker, declarative_base
3 from sqlalchemy.pool import NullPool
4 from app.config import settings
5 import logging
6
7 logger = logging.getLogger(__name__)
8
9 # Create engine with connection pooling
10 engine = create_engine(
11     settings.DATABASE_URL,
12     poolclass=NullPool if settings.DEBUG else None,
13     echo=settings.DEBUG,
14     connect_args={"check_same_thread": False} if "sqlite" in settings.DATABASE_URL else {}
15 )
16
17 SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
18 Base = declarative_base()
19
20
21 AWS: Add Debug Configuration | AWS: Edit Debug Configuration
22 def get_db():
23     """Dependency to get database session"""
24     db = SessionLocal()
25     try:
26         yield db
27     finally:
28         db.close()
29
30 AWS: Add Debug Configuration | AWS: Edit Debug Configuration
31 async def init_db():
32     """Initialize database tables"""
33     try:
34         Base.metadata.create_all(bind=engine)
35         logger.info("Database tables created successfully")
36     except Exception as e:
37         logger.error(f"Error initializing database: {e}")
38         raise
```

- **Framework / API:** FastAPI
- **Database ORM:** SQLAlchemy
- **Database:** PostgreSQL
- **Programming Language:** Python

Database Layer Architecture

The database.py service file implements six core capabilities that form the backbone of all data persistence:

1. **Connection Engine** SQLAlchemy creates a persistent connection pool between FastAPI and PostgreSQL using the DATABASE_URL from .env.
2. **Session Management** Each HTTP request gets its own database session (temporary workspace). Sessions are automatically committed or rolled back and then closed.
3. **ORM Mapping** Python classes (User, Conversation, Message, Analytics) are mapped to database tables. No raw SQL needed — queries are written in Python.
4. **Base Class** declarative_base() creates the schema foundation. All model classes inherit from this Base to register with the ORM.
5. **Auto Table Creation** init_db() runs on startup and creates all tables if they do not already exist — zero manual migrations needed in development.
6. **Connection Pooling** Connections are reused across requests instead of being created and destroyed per request — dramatically reducing latency under load.

Open Source & Community:

- Free to use
- Well-documented
- Large community support
- Regular security updates

Performance:

- Optimized query execution
- Intelligent query planner

- Connection pooling support
- Excellent for reporting

Advanced Features:

- Full-text search capabilities
- JSON/JSONB data type support
- Supports complex queries
- Advanced indexing strategies

Purpose:

It acts as the bridge between your FastAPI backend and PostgreSQL database

2. Redis

Redis is an in-memory data store with microsecond read/write latency. It serves as a critical performance layer between FastAPI and PostgreSQL, drastically reducing database load and enabling features like rate limiting, session tracking, and AI response deduplication.

Redis is implemented for the caching purpose , acting as “short term memory” . It stores user communication for a specific periods as a cach to ensure a fast response and a better user experience . For this we use API for getting access from **Render.com website**

Redis Capabilities Used

Redis Feature	How It Is Used
Key-Value Storage	Cache conversation history, messages, and AI responses
TTL / Expiration	Automatic cleanup — conversations cached for 1 hour, AI responses for 24 hours
Pub/Sub Channels	Future: real-time notifications and live chat updates
Rate Limiting	Track requests per IP per minute to prevent API abuse
Session Storage	Active user sessions stored with 30-minute expiry

Redis Usage in Application

1. Conversation Caching

```
# Cache conversation history
cache.set(f"conversation:{conv_id}", conv_data, ttl=3600)

# Retrieve from cache
cached_conv = cache.get(f"conversation:{conv_id}")
```

Purpose: Reduce database queries for frequently accessed conversations

2. Session Storage

```
# Store user session
```

```
cache.set(f"session:{user_id}:{token}", user_data, ttl=1800)
```

Purpose: Track active user sessions

3. Rate Limiting

Track request count

```
cache.incr(f"requests:{ip_address}:{minute}")
cache.expire(f"requests:{ip_address}:{minute}", 60)
```

Purpose: Prevent API abuse

4. AI Response Caching

Cache AI responses

```
cache.set(f"ai_response:{message_hash}", response, ttl=86400)
```

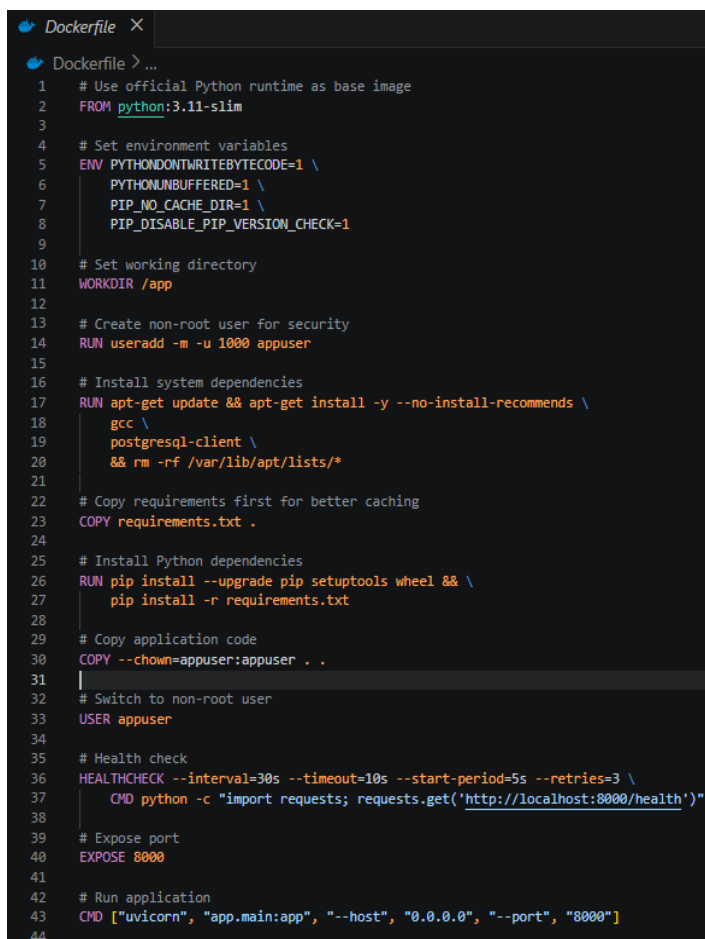
Purpose: Avoid duplicate API calls

3. Docker File

The entire application stack is containerized using Docker. This ensures that the application runs identically in development, testing, and production environments — eliminating 'works on my machine' problems entirely.

Dockerfile setup

Dockerfile:



```
Dockerfile X
Dockerfile > ...
1  # Use official Python runtime as base image
2  FROM python:3.11-slim
3
4  # Set environment variables
5  ENV PYTHONDONTWRITEBYTECODE=1 \
6      PYTHONUNBUFFERED=1 \
7      PIP_NO_CACHE_DIR=1 \
8      PIP_DISABLE_PIP_VERSION_CHECK=1
9
10 # Set working directory
11 WORKDIR /app
12
13 # Create non-root user for security
14 RUN useradd -m -u 1000 appuser
15
16 # Install system dependencies
17 RUN apt-get update && apt-get install -y --no-install-recommends \
18     gcc \
19     postgresql-client \
20     && rm -rf /var/lib/apt/lists/*
21
22 # Copy requirements first for better caching
23 COPY requirements.txt .
24
25 # Install Python dependencies
26 RUN pip install --upgrade pip setuptools wheel && \
27     pip install -r requirements.txt
28
29 # Copy application code
30 COPY --chown=appuser:appuser . .
31
32 # Switch to non-root user
33 USER appuser
34
35 # Health check
36 HEALTHCHECK --interval=30s --timeout=10s --start-period=5s --retries=3 \
37     CMD python -c "import requests; requests.get('http://localhost:8000/health')"
38
39 # Expose port
40 EXPOSE 8000
41
42 # Run application
43 CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
44
```

A Docker file is setup because to initialize all components during deployment so that it can set up the docker container.

The Dockerfile builds the FastAPI application into a lightweight, secure, production-ready container image. Here is what each step does:

- a) **Base Image** python:3.11-slim — a minimal Linux environment with Python 3.11 pre-installed. Slim variant reduces image size significantly.
- b) **Environment Variables** PYTHONDONTWRITEBYTECODE=1 and PYTHONUNBUFFERED=1 optimize Python for containerized execution (no .pyc files, instant log output).
- c) **Non-Root User** A dedicated appuser account (UID 1000) is created. The application runs as this user — not root — for security hardening.
- d) **System Dependencies** gcc and postgresql-client are installed for compiling native Python extensions and database connectivity.
- e) **Dependency Installation** requirements.txt is copied first (before source code) to leverage Docker's layer cache — only reinstalls packages when requirements change.
- f) **Health Check** Built-in HEALTHCHECK pings /health every 30 seconds. Docker marks the container unhealthy if it fails 3 times — enabling auto-restart policies.
- g) **Startup Command** uvicorn app.main:app --host 0.0.0.0 --port 8000 starts the ASGI server, making the API accessible on all network interfaces.

Docker Compose.yml

docker-compose.yml defines and orchestrates the entire production infrastructure in a single file. Running docker compose up -d brings up all four services with correct networking, volumes, and dependency ordering.

I have created docker-compose.yml file because it allows the entire setup to run with a single command: **docker-compose up** . This file automatically handles running the Dockerfile and performing health checks on tasks

build-deploy.yml :

```
build-deploy.yml 1, M X
.github > workflows > build-deploy.yml
1  name: Build and Deploy
2
3  on:
4    push:
5      branches: [ main, develop ]
6    pull_request:
7      branches: [ main, develop ]
8
9  env:
10   REGISTRY: ghcr.io
11
12  jobs:
13
14    build:
15      runs-on: ubuntu-latest
16      permissions:
17        contents: read
18        packages: write
19
20      steps:
21        - name: Checkout code
22          uses: actions/checkout@v4
23
24        # FIX: github.repository can have uppercase letters which Docker doesn't allow
25        - name: Set lowercase image name
26          id: image
27          run: echo "name=${{ env.REGISTRY }}/${{ github.repository }} | tr '[:upper:]' '[:lower:]'" >> $GITHUB_OUTPUT
28
29        - name: Set up Docker Buildx
30          uses: docker/setup-buildx-action@v3
31
32        - name: Log in to Container Registry
33          if: github.event_name != 'pull_request'
34          uses: docker/login-action@v3
35          with:
36            registry: ${{ env.REGISTRY }}
37            username: ${{ github.actor }}
38            password: ${{ secrets.GITHUB_TOKEN }}
39
40        - name: Extract metadata
41          id: meta
42          uses: docker/metadata-action@v5
43          with:
44            images: ${{ steps.image.outputs.name }}
45            tags: |
46              type=ref,event=branch
47              type=semver,pattern={{version}}
48              type=semver,pattern={{major}}.{{minor}}
49              type=sha
50
51        - name: Build and push Docker image
52          uses: docker/build-push-action@v5
53          with:
54            context: .
55            push: ${{ github.event_name != 'pull_request' }}
56            tags: ${{ steps.meta.outputs.tags }}
57            labels: ${{ steps.meta.outputs.labels }}
58            cache-from: type=registry,ref=${{ steps.image.outputs.name }}:buildcache
59            cache-to: type=registry,ref=${{ steps.image.outputs.name }}:buildcache,mode=max
```

We configured the workflow to automatically start whenever code is pushed or a pull request is created on the main or develop branches. We defined ghcr.io as the container registry so that our Docker images can be stored securely in GitHub Container Registry.

In the build section, we first checked out the project source code from the repository into the GitHub Actions runner machine. After that, we converted the repository name into lowercase because Docker image names do not support uppercase characters. Then we configured Docker Buildx to enable advanced Docker image building features like caching and optimized builds. We authenticated with GitHub Container Registry using GitHub secrets so the workflow could securely push Docker images.

Next, we extracted Docker image metadata and automatically generated image tags based on branch names, semantic versions, and commit SHA values. Then we built the Docker image of the application and pushed it to GitHub Container Registry. We also enabled Docker layer caching to improve build speed in future workflow executions.

```

62 test:
63   runs-on: ubuntu-latest
64   services:
65     postgres:
66       image: postgres:15-alpine
67       env:
68         POSTGRES_PASSWORD: postgres
69         POSTGRES_DB: test_db
70       options: >-
71         --health-cmd pg_isready
72         --health-interval 10s
73         --health-timeout 5s
74         --health-retries 5
75       ports:
76         - 5432:5432
77
78     redis:
79       image: redis:7-alpine
80       options: >-
81         --health-cmd "redis-cli ping"
82         --health-interval 10s
83         --health-timeout 5s
84         --health-retries 5
85       ports:
86         - 6379:6379
87
88   steps:
89     - name: Checkout code
90       uses: actions/checkout@v4
91
92     - name: Set up Python
93       uses: actions/setup-python@v4
94       with:
95         python-version: '3.11'
96         cache: 'pip'
97
98     - name: Install dependencies
99       run: |
100         python -m pip install --upgrade pip
101         pip install -r requirements.txt
102         pip install pytest pytest-asyncio pytest-cov
103
104     - name: Run tests
105       env:
106         DATABASE_URL: postgresql://postgres:postgres@localhost:5432/test_db
107         REDIS_URL: redis://localhost:6379/0
108         SECRET_KEY: test-secret-key
109         GEMINI_API_KEY: ${ secrets.GEMINI_API_KEY }
110         AI_MODEL: gemini-1.5-flash
111         DEBUG: "True"
112         ENVIRONMENT: test
113       run: |
114         pytest --cov=app --cov-report=xml -v || true
115
116     - name: Upload coverage to Codecov
117       uses: codecov/codecov-action@v3
118       with:
119         file: ./coverage.xml
120         fail_ci_if_error: false

```

In the testing section, we created temporary PostgreSQL and Redis service containers so that the application could be tested in a real backend environment. We configured PostgreSQL with a test database and added health checks to ensure the database service was fully ready before executing tests. Similarly, we started a Redis container and verified its availability using Redis ping health checks.

After setting up the backend services, we installed Python 3.11 and all required project dependencies including testing libraries such as Pytest, Pytest-Asyncio, and Pytest-Cov. Then we executed automated tests for the application while passing environment variables like database URL, Redis URL, API keys, debug mode, and application environment. We also generated a code coverage report and uploaded it to Codecov for coverage analysis

```

118     with:
119       file: ./coverage.xml
120       fail_ci_if_error: false
121
122   deploy:
123     needs: [build, test]
124     runs-on: ubuntu-latest
125     if: github.ref == 'refs/heads/main' && github.event_name == 'push'
126
127     steps:
128       - name: Checkout code
129         uses: actions/checkout@v4
130
131       # FIX: Use printf to preserve multi-line SSH key exactly - echo mangles newlines
132       - name: Setup SSH
133         run: |
134           mkdir -p ~/.ssh
135           printf '%s\n' "${{ secrets.DEPLOY_KEY }}" > ~/.ssh/deploy_key
136           chmod 600 ~/.ssh/deploy_key
137           ssh-keyscan -H "${{ secrets.DEPLOY_HOST }}" >> ~/.ssh/known_hosts
138           ssh-keygen -l -f ~/.ssh/deploy_key
139
140       - name: Deploy to EC2
141         run: |
142           ssh -i ~/.ssh/deploy_key "${{ secrets.DEPLOY_USER }}"@${{ secrets.DEPLOY_HOST }} \
143             "cd ${{{ secrets.DEPLOY_PATH }}} && \
144             git pull origin main && \
145             docker compose up -d --build && \
146             docker compose exec -T app alembic upgrade head || true && \
147             docker compose ps"
148
149       - name: Health check
150         run: |
151           echo "Waiting for app to be ready..."
152           for i in {1..30}; do
153             # Use http (port 80 via nginx), -k skips SSL verify, remove || echo to avoid double-000
154             # -k = skip SSL verify (self-signed cert), -L = follow 301 redirect http->https
155             STATUS=$(curl -sk -o /dev/null -w "%{http_code}" https://${{ secrets.DEPLOY_HOST }}/health 2>/dev/null)
156             echo "Attempt $i/30 - status: $STATUS"
157             if [ "$STATUS" = "200" ]; then
158               echo "🟢 Akanksha You are great and the deployment was successful!"
159               exit 0
160             fi
161             sleep 10
162           done
163           echo "❌ tata by by failed"
164           exit 1
165
166

```

In the deployment section, we configured the deployment job to run only after the build and testing jobs were completed successfully. We restricted deployment so that it only happens when code is pushed to the main branch. Then we securely connected to the AWS EC2 server using SSH authentication and deployment keys stored in GitHub Secrets.

Once connected to the server, we pulled the latest code from GitHub, rebuilt Docker containers using Docker Compose, started all services in detached mode, and executed Alembic database migrations to update the PostgreSQL database schema automatically. Finally, we performed a health check by continuously calling the application's /health endpoint until the application returned HTTP status code 200, confirming that the deployment was successful and the application was running correctly.

Service	Image	Ports	Role
db	postgres:15-alpine	5432:5432	Relational database with persistent volume and health checks
cache	redis:7-alpine	6379:6379	In-memory cache with password protection and AOF persistence
app	ai-chat-app (built)	8000:8000	FastAPI backend — waits for db and cache to be healthy before starting
nginx	nginx:alpine	80:80, 443:443	Reverse proxy with SSL, CORS headers, and upstream to app:8000

4. Ci/CD pipeline

The project implements a full three-stage CI/CD pipeline using GitHub Actions (build-deploy.yml). Every push to main or develop triggers the pipeline automatically — from code to production with zero manual steps.

Pipeline Overview

Stage	Trigger	What Happens
Build	Push to main or develop	Docker image built, tagged, and pushed to GitHub Container Registry (ghcr.io)
Test	Push to main or develop	Python tests run against live PostgreSQL + Redis containers
Deploy	Push to main only	SSH into EC2, pull latest code, restart Docker containers, run DB migrations, verify health

Build Process :-

The build section is responsible for creating a production-ready Docker image of the application and storing it in the container registry.

It automates the application packaging process for deployment.

What We Have in Build Section

1. Ubuntu Runner Environment : The build job runs on: **ubuntu-latest**

Purpose : Provides a fresh Linux environment where the build process executes automatically.

2. Repository Code Checkout : The workflow downloads the latest source code from GitHub repository.

Purpose : Ensures the pipeline uses the most recent application code for building.

3. Docker Buildx Setup : Docker Buildx is configured for advanced Docker image building.

Purpose:

- enables optimized Docker builds
- supports caching
- improves build performance
- allows scalable image creation

4. Container Registry Authentication : The workflow securely logs into GitHub Container Registry (ghcr.io).

Purpose:

- allows pushing Docker images
- secures container access
- enables image storage and retrieval

Security is managed using GitHub Secrets and Tokens.

5. Metadata Extraction

Docker image metadata is automatically generated.

Generated Metadata Includes:

- branch tags
- semantic version tags
- commit SHA tags

Purpose:

- version management
- release tracking
- rollback support
- image identification

6. Docker Image Build : The application Docker image is built using the project Dockerfile.

Purpose: Creates a portable containerized application package.

7. Docker Image Push : The built Docker image is pushed to GitHub Container Registry.

Purpose: Makes the image available for deployment on servers or cloud infrastructure.

8. Docker Layer Caching: The workflow uses Docker cache layers.

Purpose:

- speeds up future builds
- reduces build time
- optimizes CI/CD performance

Test process :-

What We Have in Test Section

1. Ubuntu Runner Environment

The test job also runs on: **ubuntu-latest**

Purpose: Provides isolated environment for automated testing.

2. PostgreSQL Test Service

A temporary PostgreSQL container is created.

Purpose:

- tests database integration
- validates database queries
- simulates production database environment

Features:

- PostgreSQL 15 Alpine image
- health monitoring
- isolated test database

3. Redis Test Service

A temporary Redis container is created.

Purpose:

- tests caching system
- validates Redis communication
- checks temporary data handling

Features:

- Redis 7 Alpine image
- health checks
- isolated cache environment

4. Repository Code Checkout

The latest project code is downloaded.

Purpose: Allows test execution on current application version.

5. Python Environment Setup

Python 3.11 runtime is installed automatically.

Purpose: Creates Python environment required for backend execution.

Additional Feature:

- pip dependency caching

Benefits:

- faster workflow execution
- reduced installation time

6. Dependency Installation

The workflow installs:

- project dependencies
- pytest
- pytest-asyncio
- pytest-cov

Purpose: Provides testing frameworks and required backend libraries.

7. Automated Test Execution

Automated tests are executed using pytest.

Testing Includes:

- API testing
- database testing
- Redis testing
- backend logic validation
- async functionality testing

Purpose: Detects bugs and validates application stability.

8. Environment Variable Configuration

Temporary environment variables are created during testing.

Includes:

- DATABASE_URL
- REDIS_URL
- SECRET_KEY

Purpose: Simulates application runtime environment during tests.

9. Code Coverage Report Generation

The workflow generates a test coverage report.

Purpose: Measures how much application code is covered by automated tests.

10. Coverage Upload to Codecov

Coverage reports are uploaded to Codecov.

Purpose: Provides centralized code quality and testing analytics.

Deploy Process

What We Have in Deploy Section

1. Dependency on Build and Test Jobs

The deploy job depends on: **build + test**

Purpose: Deployment starts only after successful build and testing.

2. Main Branch Deployment Restriction

Deployment runs only for: **main branch push events**

Purpose: Prevents accidental deployment from development branches.

3. Repository Code Checkout

The latest repository code is downloaded again.

Purpose: Ensures deployment uses updated application configuration.

4. Secure SSH Authentication

The workflow securely connects to production server using SSH.

Uses:

- DEPLOY_KEY
- DEPLOY_HOST
- DEPLOY_USER
- DEPLOY_PATH

stored inside GitHub Secrets.

Purpose: Provides secure automated server access.

5. SSH Key Configuration

The workflow:

- creates SSH key file
- sets secure permissions
- adds server host verification

Purpose: Establishes trusted and secure server communication.

6. Remote Server Connection

The workflow connects to VPS/server remotely using SSH.

Purpose: Allows automated deployment execution on production infrastructure.

7. Pull Latest Source Code

The server pulls latest application code from GitHub repository.

Purpose: Updates production environment with latest backend version.

8. Pull Latest Docker Images

The deployment process downloads newest Docker images.

Purpose: Ensures server runs latest application containers.

9. Restart Docker Containers

Docker Compose restarts application services.

Purpose: Applies latest updates without manual server intervention.

10. Database Migration Execution

Alembic database migrations are executed automatically.

Purpose: Updates production database schema safely during deployment.

Benefits:

- schema synchronization
- automated database updates
- version-controlled database structure

11. Application Health Check

After deployment, the workflow checks: **/health endpoint**

Purpose: Verifies that deployed application is running successfully.

12. Deployment Validation

The workflow retries health checks multiple times.

Success Result: Deployment successful

Failure Result: Deployment health check failed

Purpose:

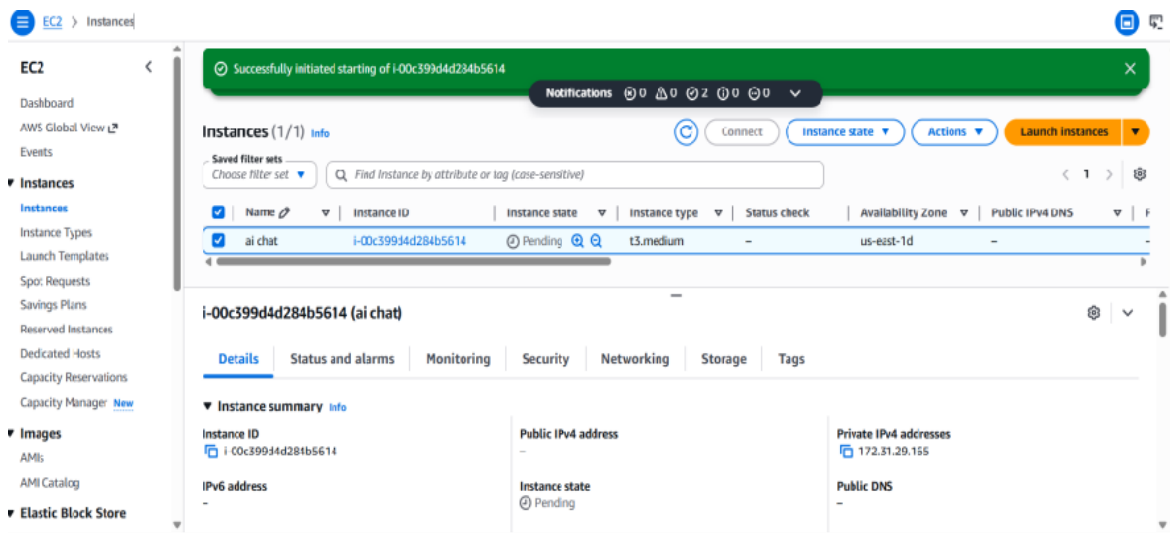
Automatically detects deployment failures and improves production reliability.

6.2 AWS EC2 Instance

The application is deployed on an AWS EC2 instance running Ubuntu 22.04 LTS. The EC2 server hosts all Docker containers behind NGINX and connects to managed PostgreSQL and Redis services on Render.com for persistence.

EC2 Instance Configuration

Setting	Value	Reason
Instance Type	t3.medium	4 GB RAM needed for Docker + all 4 containers
AMI	Ubuntu 22.04 LTS	Long-term support, Docker-compatible, widely documented
Storage	20 GB SSD	Sufficient for OS, Docker images, and application code
Security Group	Ports 22, 80, 443	SSH access + HTTP/HTTPS for the web application
Key Pair	ai-chat.pem	ED25519 key for secure passwordless SSH authentication
Availability Zone	us-east-1d	AWS Free Tier region t3.medium



Step 1 — EC2 Provisioning

I provisioned an AWS EC2 instance using Ubuntu Server 22.04 LTS with a t2.micro configuration and 20 GB storage to host the AI backend application in a cloud environment.

Step 2 — Configure Security Group

I configured the EC2 Security Group by allowing only essential ports such as SSH (22), HTTP (80), and HTTPS (443) to ensure secure remote access and controlled web traffic.

Open these ports:

Port	Protocol	Purpose
22	SSH	Remote server access
80	HTTP	Website traffic
443	HTTPS	Secure website traffic

Block all other ports to improve server security.

Step 3 — SSH Key Management

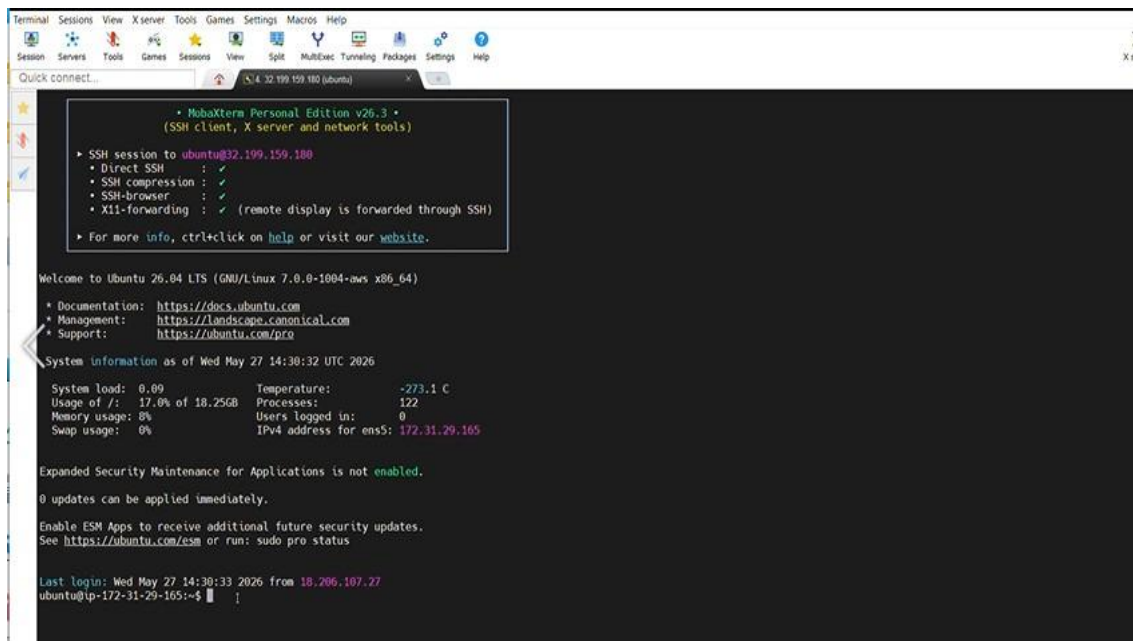
I generated and configured an SSH key pair for secure authentication and encrypted remote access to the EC2 server..

Step 4 — Remote Server Access

I established a secure SSH connection to the EC2 instance using MobaXterm and authenticated through the private .pem deployment key.

Create a new SSH session with these details:

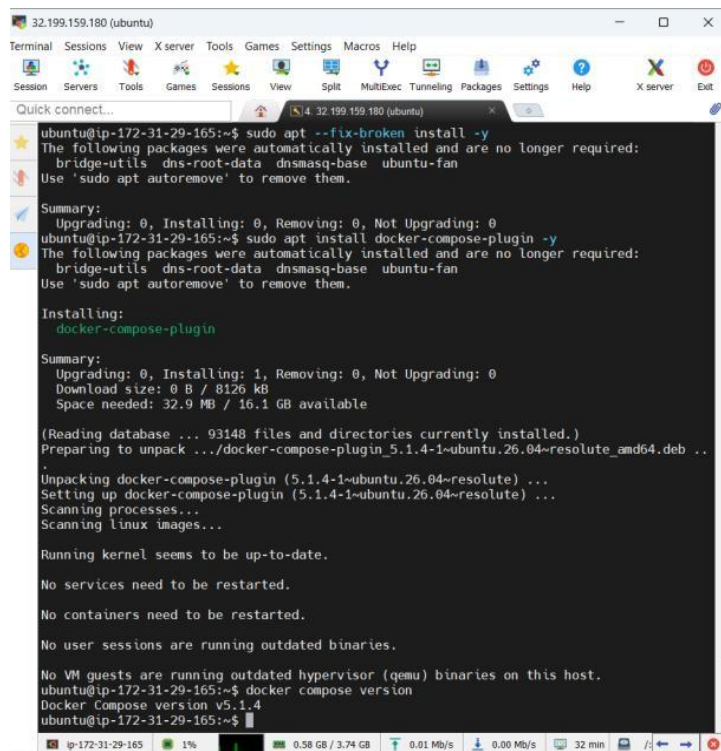
Field	Value
Remote Host	EC2 Public IP
Port	22
Username	ubuntu
SSH Key	ai-chat.pem



```

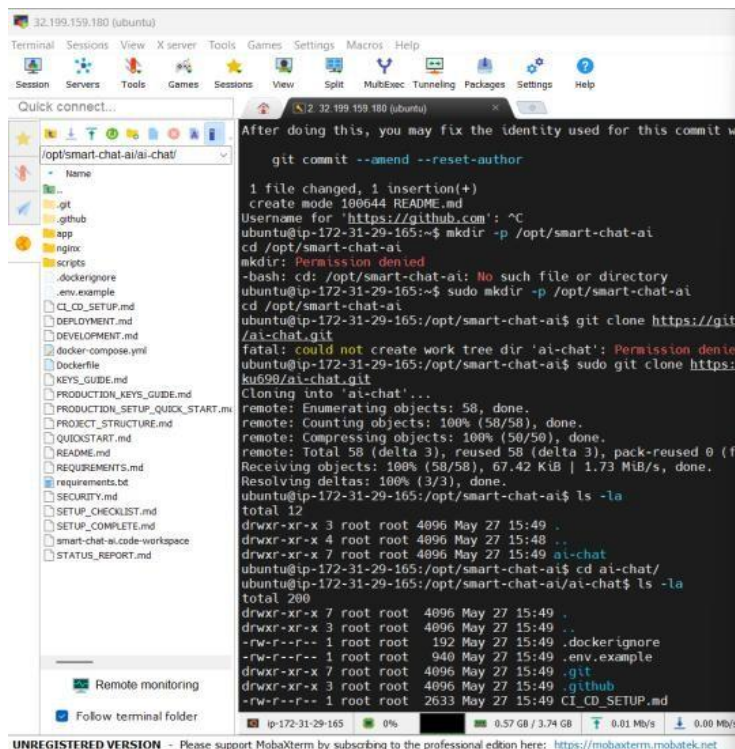
apt          apt-cache      apt-config   apt-mark
apt-add-repository apt-cdrom    apt-get
ubuntu@ip-172-31-29-165:~$ sudo apt-get update
Get:1 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute InRelease [136 kB]
Get:2 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute-updates InRelease [136 kB]
Get:3 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute-backports InRelease [136 kB]
Get:4 http://security.ubuntu.com/ubuntu resolute-security InRelease [136 kB]
Get:5 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute/main amd64v3 Packages [1483 kB]
Get:6 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute/main Translation-en [524 kB]
Get:7 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute/main amd64 Components [395 kB]

```



Step 5 — Docker Environment Setup

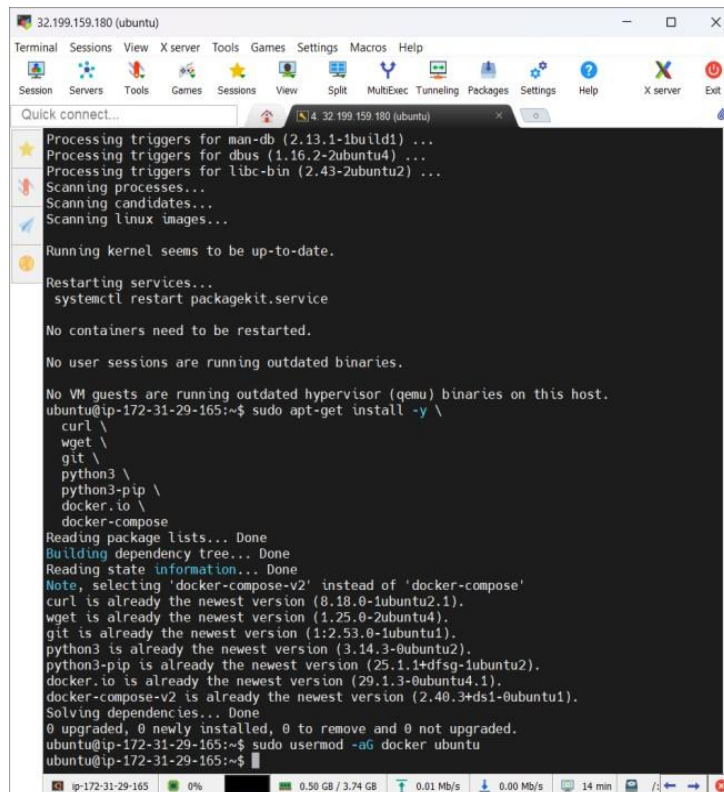
I installed Docker, Docker Compose, and Git on the EC2 server to enable containerized deployment, multi-container orchestration, and source code management.



The screenshot shows a MobaXterm window with a terminal session on an Ubuntu system. The terminal displays the following commands and output:

```
git commit --amend --reset-author
1 file changed, 1 insertion(+)
create mode 100644 README.md
Username for 'https://github.com': ^C
ubuntu@ip-172-31-29-165:~$ mkdir -p /opt/smart-chat-ai
cd /opt/smart-chat-ai
mkdir: Permission denied
-bash: cd: /opt/smart-chat-ai: No such file or directory
ubuntu@ip-172-31-29-165:~$ sudo mkdir -p /opt/smart-chat-ai
cd /opt/smart-chat-ai
ubuntu@ip-172-31-29-165:/opt/smart-chat-ai$ git clone https://git
/ai-chat.git
fatal: could not create work tree dir 'ai-chat': Permission denied
ubuntu@ip-172-31-29-165:/opt/smart-chat-ai$ sudo git clone https:
ku690/ai-chat.git
Cloning into 'ai-chat'...
remote: Enumerating objects: 58, done.
remote: Counting objects: 100% (58/58), done.
remote: Compressing objects: 100% (50/50), done.
remote: Total 58 (delta 3), reused 58 (delta 3), pack-reused 0 (f
Receiving objects: 100% (58/58), 67.42 KiB | 1.73 MiB/s, done.
Resolving deltas: 100% (3/3), done.
ubuntu@ip-172-31-29-165:/opt/smart-chat-ai$ ls -la
total 12
drwxr-xr-x 3 root root 4096 May 27 15:49 .
drwxr-xr-x 4 root root 4096 May 27 15:48 ..
drwxr-xr-x 7 root root 4096 May 27 15:49 ai-chat
ubuntu@ip-172-31-29-165:/opt/smart-chat-ai$ cd ai-chat/
ubuntu@ip-172-31-29-165:/opt/smart-chat-ai/ai-chat$ ls -la
total 200
drwxr-xr-x 7 root root 4096 May 27 15:49 .
drwxr-xr-x 3 root root 4096 May 27 15:49 ..
-rw-r--r-- 1 root root 192 May 27 15:49 .dockerignore
-rw-r--r-- 1 root root 940 May 27 15:49 .env.example
drwxr-xr-x 7 root root 4096 May 27 15:49 .git
drwxr-xr-x 3 root root 4096 May 27 15:49 .github
-rw-r--r-- 1 root root 2633 May 27 15:49 CI_CD_SETUP.md
```

At the bottom of the window, there is a notice: "UNREGISTERED VERSION - Please support MobaXterm by subscribing to the professional edition here: <https://mobaxterm.mobatek.net>"



The screenshot shows a MobaXterm window with a terminal session on an Ubuntu system. The terminal displays the following commands and output:

```
Processing triggers for man-db (2.13.1-1build1) ...
Processing triggers for dbus (1.16.2-2ubuntu4) ...
Processing triggers for libc-bin (2.43-2ubuntu2) ...
Scanning processes...
Scanning candidates...
Scanning linux images...
Running kernel seems to be up-to-date.
Restarting services...
systemctl restart packagekit.service
No containers need to be restarted.
No user sessions are running outdated binaries.
No VM guests are running outdated hypervisor (qemu) binaries on this host.
ubuntu@ip-172-31-29-165:~$ sudo apt-get install -y \
curl \
wget \
git \
python3 \
python3-pip \
docker.io \
docker-compose
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Note, selecting 'docker-compose-v2' instead of 'docker-compose'
curl is already the newest version (8.18.0-1ubuntu2.1).
wget is already the newest version (1.25.0-2ubuntu4).
git is already the newest version (1:2.53.0-1ubuntu1).
python3 is already the newest version (3.14.3-0ubuntu2).
python3-pip is already the newest version (25.1.1+dfsg-1ubuntu2).
docker.io is already the newest version (29.1.3-0ubuntu4.1).
docker-compose-v2 is already the newest version (2.40.3+ds1-0ubuntu1).
Solving dependencies... Done
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
ubuntu@ip-172-31-29-165:~$ sudo usermod -aG docker ubuntu
ubuntu@ip-172-31-29-165:~$
```

Step 6 — Docker Permission Management

I configured non-root Docker access by adding the Ubuntu user to the Docker group, enabling secure and streamlined container management without sudo privileges.

Step 7 — Application Deployment Setup

In this step, I created a dedicated deployment directory inside the Linux server using the `mkdir` command and navigated into it using `cd`. After setting up the directory structure, I cloned the application source code from the GitHub repository using `git`, which downloaded all project files into the EC2 server for deployment. I also verified the downloaded files using the `ls -la` command and accessed the project folder to continue the deployment process.

```
sudo mkdir -p /opt/smart-chat-ai
cd /opt/smart-chat-ai
git clone https://github.com/Akku690/ai-chat.git
sudo git clone https://github.com/Akku690/ai-chat.git
ls -la
cd ai-chat/
```

I used **`docker-compose up -d`** to start all application services and containers in detached mode for background execution. To verify running services and container health, I used **`docker-compose ps`**.

For monitoring application activity and debugging runtime issues, I viewed live container logs using **`docker-compose logs -f app`**. To temporarily stop all running services without removing containers, I used **`docker-compose stop`**.

For complete cleanup of containers, networks, and persistent volumes, I executed **`docker-compose down -v`**. Finally, I rebuilt all Docker images from scratch without using cache through **`docker-compose build --no-cache`** to ensure fresh application builds and updated dependencies.

```
docker compose up -d
```

```
docker compose ps
```

Step 8 — Environment Configuration

I configured the production environment variables by creating a `.env` file from the sample configuration template. I then updated the file with required production credentials such as API keys, database connection URLs, authentication secrets, and external service configurations to securely manage application settings during deployment.

Configure Environment Variables

```
cp .env.example .env
nano .env

# Required settings:
DEBUG=False
ENVIRONMENT=production
SECRET_KEY=<your-secret-key>

# PostgreSQL (Render.com)
DB_HOST=dpg-xxxx.ohio-postgres.render.com
DB_PORT=5432
DB_USER=ai_yyfn_user
DB_NAME=ai_yyfn

# Redis (Render.com)
REDIS_HOST=ohio-keyvalue.render.com
REDIS_PORT=6379
REDIS_SSL=True

# Gemini AI
GEMINI_API_KEY=<your-gemini-key>
```

AI_MODEL=gemini-pro

Step 9 — Application Deployment and Verification

I launched the application using Docker Compose to start all backend services and containers in detached mode for background execution. After deployment, I verified the container status to ensure that all application services were running successfully and operational.

```
# Build and start all services in detached mode
docker compose up -d

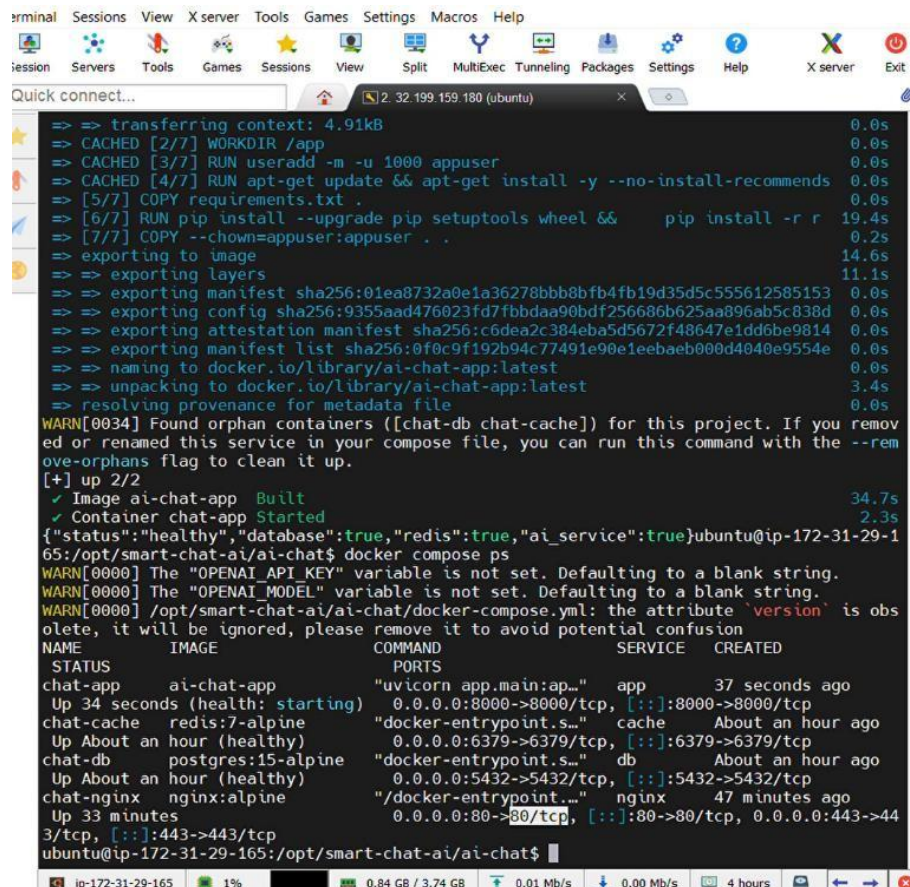
# Verify all containers are running
docker compose ps

# Test the health endpoint
curl http://localhost:8000/health
# Expected: {"status":"healthy","database":true,"redis":true,"ai_service":true}
```

Step 10 — Application Health Verification

I verified the application deployment by sending a request to the /health endpoint using the curl command. This confirmed that the backend services, Docker containers, and API endpoints were running successfully and responding correctly in the production environment.

Now verifying the health

A screenshot of a terminal window with a dark background and light-colored text. The window title is "Terminal". The terminal shows the output of a Docker Compose command. It starts with "transferring context: 4.91kB", followed by several "cached" messages for WORKDIR, useradd, apt-get, COPY, and RUN commands. Then it shows "exporting to image" and "exporting layers". After that, it shows "exporting manifest" and "exporting config" for the ai-chat-app. The output then shows "naming to docker.io/library/ai-chat-app:latest" and "unpacking to docker.io/library/ai-chat-app:latest". A warning message appears: "WARN[0034] Found orphan containers ([chat-db chat-cache]) for this project. If you remove or renamed this service in your compose file, you can run this command with the --remove-orphans flag to clean it up." The output then shows "up 2/2" and "Image ai-chat-app Built 34.7s". Finally, it shows "Container chat-app Started 2.3s". Below this, the output of "docker compose ps" is shown, listing the services: chat-app, chat-cache, chat-db, and chat-nginx, along with their images, commands, ports, and status. The chat-app is shown as "starting" and the others as "healthy". The terminal also shows the command "docker compose ps" and the output of "docker compose ps".

```
=> => transferring context: 4.91kB 0.0s
=> CACHED [2/7] WORKDIR /app 0.0s
=> CACHED [3/7] RUN useradd -m -u 1000 appuser 0.0s
=> CACHED [4/7] RUN apt-get update && apt-get install -y --no-install-recommends 0.0s
=> [5/7] COPY requirements.txt . 0.0s
=> [6/7] RUN pip install --upgrade pip setuptools wheel && pip install -r r 19.4s
=> [7/7] COPY --chown=appuser:appuser . . 0.2s
=> exporting to image 14.6s
=> exporting layers 11.1s
=> exporting manifest sha256:01ea8732a0e1a36278bbb8bfb4fb19d35d5c555612585153 0.0s
=> exporting config sha256:9355aad476023fd7fbbdaa90bdf256686b625aa896ab5c838d 0.0s
=> exporting attestation manifest sha256:c6dea2c384eba5d5672f48647e1dd6ba9814 0.0s
=> exporting manifest list sha256:0f0c9f192b94c77491e90e1eebaeb000d4040e9554e 0.0s
=> naming to docker.io/library/ai-chat-app:latest 0.0s
=> unpacking to docker.io/library/ai-chat-app:latest 3.4s
=> resolving provenance for metadata file 0.0s
WARN[0034] Found orphan containers ([chat-db chat-cache]) for this project. If you remove
or renamed this service in your compose file, you can run this command with the --rem
ove-orphans flag to clean it up.
[+] up 2/2
✓ Image ai-chat-app Built 34.7s
✓ Container chat-app Started 2.3s
{"status":"healthy","database":true,"redis":true,"ai_service":true}ubuntu@ip-172-31-29-1
65:/opt/smart-chat-ai/ai-chat$ docker compose ps
WARN[0000] The "OPENAI_API_KEY" variable is not set. Defaulting to a blank string.
WARN[0000] The "OPENAI_MODEL" variable is not set. Defaulting to a blank string.
WARN[0000] /opt/smart-chat-ai/ai-chat/docker-compose.yml: the attribute 'version' is obs
olete, it will be ignored, please remove it to avoid potential confusion
NAME          IMAGE          COMMAND          SERVICE    CREATED
STATUS
chat-app      ai-chat-app    "uvicorn app.main:ap..." app        37 seconds ago
Up 34 seconds (health: starting) 0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp
chat-cache    redis:7-alpine "docker-entrypoint.s..." cache      About an hour ago
Up About an hour (healthy) 0.0.0.0:6379->6379/tcp, [::]:6379->6379/tcp
chat-db       postgres:15-alpine "docker-entrypoint.s..." db         About an hour ago
Up About an hour (healthy) 0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp
chat-nginx    nginx:alpine   "/docker-entrypoint..." nginx      47 minutes ago
Up 33 minutes 0.0.0.0:80->80/tcp, [::]:80->80/tcp, 0.0.0.0:443->44
3/tcp, [::]:443->443/tcp
ubuntu@ip-172-31-29-165:/opt/smart-chat-ai/ai-chat$
```

Docker-based AI chat application was successfully built and deployed on the AWS EC2 server. We have verify our PostgreSQL - its running , Redis – its running . I also verified the running containers and their health status using the **docker compose ps** command.

The application is running on the following ports:

Service	Port	Purpose
chat-app	8000	Main FastAPI backend API service
nginx	80 and 443	HTTP and HTTPS reverse proxy/web traffic
postgres	5432	PostgreSQL database service
redis	6379	Redis caching and memory service

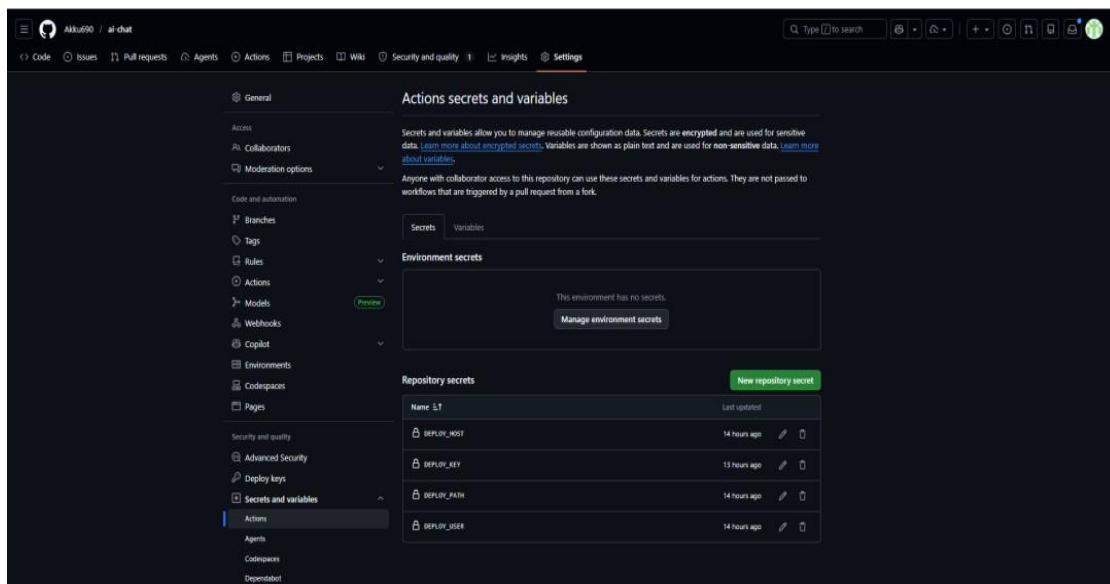
Output :

```
Not secure https://32.199.159.180
New Tab Use your camera &... Gmail YouTube W Wikipedia Maps News Translate Adobe Acrobat Driving Revenue, SC... Claude Content Player
pretty-print
{"message": "Welcome to Smart Chat API with Memory", "version": "1.0.0", "docs": "/docs", "openapi": "/openapi.json"}
```

6.3 Setting up Github For Automatic Continuous Integration and deployment

Step 1 — Configure GitHub Environment Variables

First, I configured the required environment variables and secrets in GitHub Actions settings. I navigated to the repository settings, accessed the Actions secrets and variables section, and securely added deployment credentials, API keys, SSH keys, and other production environment configurations required for the CI/CD pipeline.



Step 2 — Establish Secure EC2 and GitHub Connection

Next, I established a secure connection between the AWS EC2 server and GitHub using SSH authentication. I generated and configured SSH keys to enable secure communication and automated deployment access between the GitHub Actions workflow and the EC2 instance.

```

cat ~/.ssh/deploy_key
Generating public/private ed25519 key pair.
Your identification has been saved in /home/ubuntu/.ssh/deploy_key
Your public key has been saved in /home/ubuntu/.ssh/deploy_key.pub
The key fingerprint is:
SHA256:9s+XlIupt42ZQ7dEfizeQ07qcqt6DTp3Us2Q0fjVZos ubuntu@ip-172-31-29-165
The key's randomart image is:
+--[ED25519 256]--+
|
|      . o .
|      . = o .+
|    S . 0+=.
|      . .. oEB+=
|      .+.+=+=+
|      o =0+=XO.
|      .=.0000.
+-----[SHA256]-----+
-----BEGIN OPENSSH PRIVATE KEY-----
b3BlbnZac1rZXktZjEAAAAABG5vbmUAAAAEbm9uZQAAAAAAAAABAAAAMWAAAAATc2gtZW
QyNTUxOQAAACAOE6foZKukZcS9vj1FXQI/Qpx4BLVi9oqJd17q//V5RAAAAB73Cr+e9wq
/gAAAAATc2gtZWQyNTUxOQAAACAOE6foZKukZcS9vj1FXQI/Qpx4BLVi9oqJd17q//V5RA
AAAEcyPxuFa82pLx4RRQMKjn7Er1WanaanuYg929xtRudINA4Tp+hkpSR1xL2+PUVdAj9C
nHgEtWL2io13Xur/9X1EAAAAF3Vidw50dUBpcC0xNzItMzEtMjktMTY1AQIDBAUG
-----END OPENSSH PRIVATE KEY-----

```

Step 3 — Integrate GitHub with EC2 Deployment Workflow

After configuring SSH authentication, I integrated GitHub and the EC2 server within the CI/CD pipeline. This enabled automated deployment functionality where code changes pushed to the repository could be securely fetched and deployed directly onto the EC2 production server.

```

terminal Sessions View X server Tools Games Settings Macros Help
Session Servers Tools Games Sessions View Split MultiExec Tunneling Packages Settings Help X server Exit

Quick connect... 2. 32.199.159.180 (ubuntu)
chat-db postgres:15-alpine "docker-entrypoint.s..." db About an hour ago
Up About an hour (healthy) 0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp
chat-nginx nginx:alpine "/docker-entrypoint..." nginx 47 minutes ago
Up 33 minutes 0.0.0.0:80->80/tcp, [::]:80->80/tcp, 0.0.0.0:443->44
3/tcp, [::]:443->443/tcp
ubuntu@ip-172-31-29-165:/opt/smart-chat-ai/ai-chat$ git checkout
M app/config.py
M app/services/ai.py
M nginx/nginx.conf
M requirements.txt
Your branch is up to date with 'origin/main'.
ubuntu@ip-172-31-29-165:/opt/smart-chat-ai/ai-chat$ git add .
ubuntu@ip-172-31-29-165:/opt/smart-chat-ai/ai-chat$ git commit -u origin main
error: pathspec 'origin' did not match any file(s) known to git
error: pathspec 'main' did not match any file(s) known to git
ubuntu@ip-172-31-29-165:/opt/smart-chat-ai/ai-chat$ git commit -m "hello"
[main 8626c8e] hello
Committer: Ubuntu <ubuntu@ip-172-31-29-165.ec2.internal>
Your name and email address were configured automatically based
on your username and hostname. Please check that they are accurate.
You can suppress this message by setting them explicitly:

git config --global user.name "Your Name"
git config --global user.email you@example.com

After doing this, you may fix the identity used for this commit with:

git commit --amend --reset-author

7 files changed, 100 insertions(+), 119 deletions(-)
create mode 100644 .env
create mode 100644 nginx/ssl/cert.pem
create mode 100644 nginx/ssl/key.pem
ubuntu@ip-172-31-29-165:/opt/smart-chat-ai/ai-chat$ git push -u origin main
Username for 'https://github.com': Akku690
Password for 'https://Akku690@github.com':
remote: Invalid username or token. Password authentication is not supported for Git oper
ations.
fatal: Authentication failed for 'https://github.com/Akku690/ai-chat.git/'
ubuntu@ip-172-31-29-165:/opt/smart-chat-ai/ai-chat$

```

7. Error and Troubleshooting

Error 1. Backend crash — port 8000 not responding

Issue

While running the application on the EC2 instance, the backend service failed to start and was not accessible on port 8000. The container kept crashing on startup.

Root cause

Two conflicting API keys were declared in the environment variables — one for OpenAI and one for Gemini. The OpenAI key was added by mistake and was causing a conflict at initialization, resulting in the backend crashing before it could bind to port 8000.

Resolution

Removed the OpenAI API key from the environment variables.
Removed the reference to the OpenAI key from the configuration file.
Restarted the services using docker compose up — backend started successfully and began listening on port 8000.

Error 2. Git push failing — access denied from EC2

Issue

After making configuration file changes on the EC2 instance, pushing the updated code back to the GitHub repository failed due to an authentication error. The EC2 machine did not have the required access credentials.

Root cause

The EC2 instance lacked a valid authentication method to communicate with the GitHub repository.
GitHub no longer accepts password-based authentication for Git operations — a Personal Access Token (PAT) is required.

Resolution

Generated a GitHub Personal Access Token (PAT) with the required repository permissions.
Configured the EC2 instance to use this token as the credential when pushing to the GitHub repository.
Push succeeded — EC2 is now authorized to interact with the remote repository.

Error 3 : Redis service inactive — connection refused

Issue

The Redis service was not running and not responding to connection attempts. The application could not establish a connection to Redis, causing dependent services to fail.

Root cause

The Redis provider (external service) was configured to allow connections only from whitelisted IP addresses.

The public IP of the EC2 instance had not been added to the Redis access whitelist, causing all connection requests to be rejected.

Resolution

Retrieved the public IP of the EC2 instance.

Navigated to the Redis provider dashboard and added the EC2 public IP to the access control / IP whitelist.

Re-ran docker compose up — Redis connected successfully and the service became active.

```
# HTTPS reverse proxy
server {
    listen 443 ssl;
    http2 on;
    server_name _;

    ssl_certificate /etc/nginx/ssl/cert.pem;
    ssl_certificate_key /etc/nginx/ssl/key.pem;
    ssl_protocols TLSv1.2 TLSv1.3;

    location / {
        proxy_pass http://app:8080;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}

EOF
ubuntu@ip-172-31-29-165:/opt/smart-chat-ai/ai-chat$ mkdir -p nginx/conf.d

docker compose up -d nginx

docker compose ps
WARN[0000] /opt/smart-chat-ai/ai-chat/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
WARN[0000] Found orphan containers ([chat-db chat-cache]) for this project. If you removed or renamed this service in your compose file, you can run this command with the --remove-orphans flag to clean it up.
[*] up 2/2
✓ Container chat-app Running
✓ Container chat-nginx Running
WARN[0000] /opt/smart-chat-ai/ai-chat/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
NAME          IMAGE          COMMAND                  SERVICE    CREATED      STATUS      PORTS
chat-app      ai-chat-app    "uvicorn app.main:ap..." app        18 minutes ago Up 18 minutes (unhealthy) 0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
chat-cache    redis:7-alpine "docker-entrypoint.s..." cache      52 minutes ago Up 52 minutes (healthy) 0.0.0.0:6379->6379/tcp, [::]:6379->6379/tcp
chat-db       postgres:15-alpine "docker-entrypoint.s..." db         52 minutes ago Up 52 minutes (healthy) 0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp
chat-nginx    nginx:alpine    "/docker-entrypoint..." nginx      18 minutes ago Up 4 minutes          0.0.0.0:80->80/tcp, [::]:80->80/tcp, 0.0.0.0:443->443/tcp, [::]:443->443/tcp
ubuntu@ip-172-31-29-165:/opt/smart-chat-ai/ai-chat$
```

Error 4. Frontend crash — SSL certificate missing on port 443

Issue

The frontend began crashing immediately after deployment. It was attempting to serve traffic over HTTPS (port 443) but failing because no SSL/TLS certificate was installed on the server.

Root cause

The frontend was configured to require a verified SSL certificate to serve HTTPS traffic on port 443.

No SSL certificate had been provisioned or initialized on the EC2 server, causing the verification to fail and the frontend to crash on startup.

Resolution

Provisioned and initialized an SSL certificate on the EC2 server.

Re-ran the startup command — port 443 (HTTPS) and port 80 (HTTP) both came up successfully.

Executed a full health check across all services — every component reported a healthy status and the deployment was confirmed stable.

Error 5. CI/CD pipeline — progressive failure resolution

Three pipeline runs were required to reach a fully successful deployment. Each run exposed a distinct issue that was diagnosed, resolved, and re-triggered.

Push 1 — Build failed

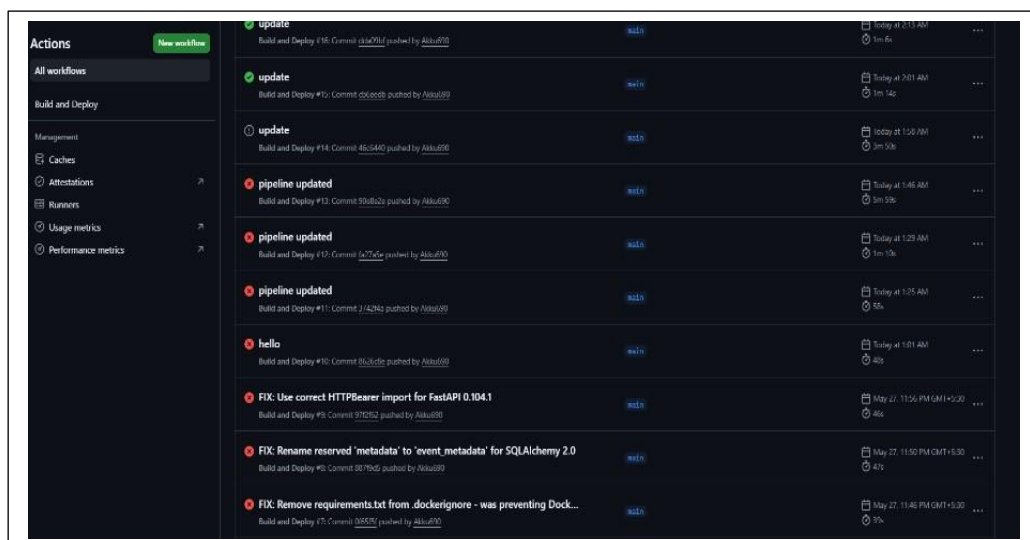
Root cause

The build failed due to a case-sensitivity error in a file name or import statement — a letter was incorrectly capitalised or lowercased.

This went undetected locally because Windows and macOS use case-insensitive file systems. The Linux-based GitHub Actions runner is case-sensitive and could not locate the file, causing the build to fail.

Resolution

- Inspected the build logs to identify the exact file or import with the incorrect casing.
- Corrected the letter casing to precisely match the actual filename on disk.
- Committed the fix and pushed to GitHub to re-trigger the pipeline



Push 2 — Deployment failed

Issue

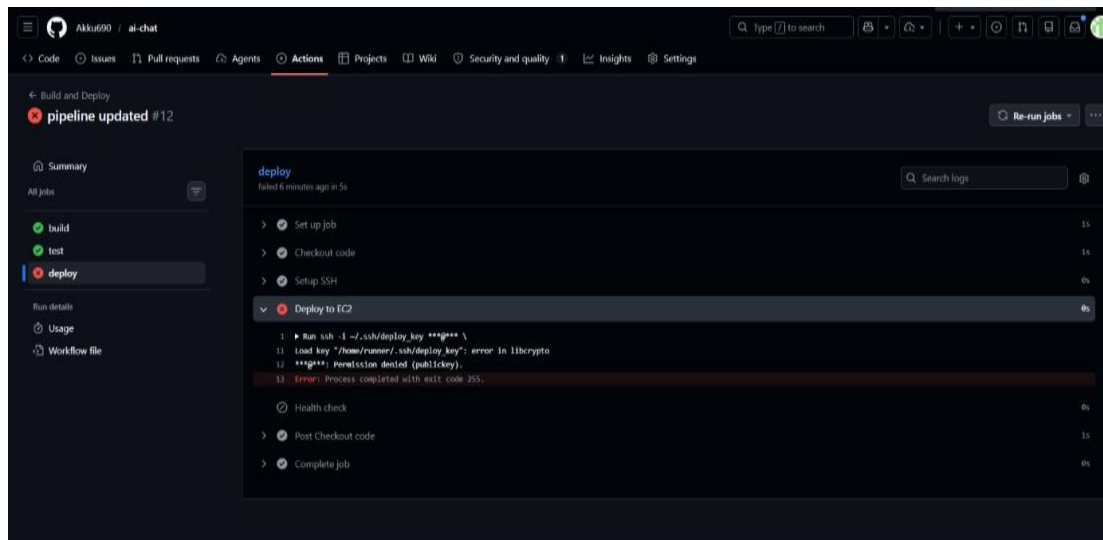
Tests and build both passed successfully, but the pipeline failed at the final deployment stage. The GitHub Actions workflow was unable to connect to and deploy onto the EC2 production server.

Root cause

- The deployment logs confirmed that GitHub Actions was being denied access to the EC2 instance — the SSH authentication was failing.
- The SSH private key stored in the GitHub Actions secret (DEPLOY_KEY) was invalid — it had been copied incorrectly, likely with missing characters, extra whitespace, or an incomplete key body.
- As a result, the runner could not authenticate with the EC2 server and the deployment step aborted with an invalid user key error.

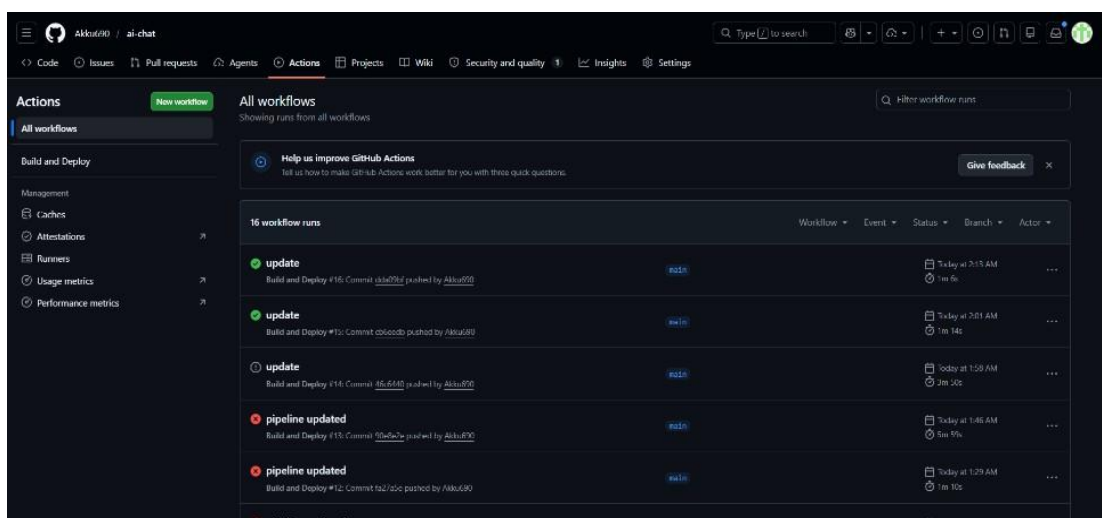
Resolution

- Reviewed the GitHub Actions deployment log and confirmed the error was invalid SSH key / permission denied.
- Went back to the EC2 instance and carefully copied the exact, complete SSH private key — including the -----BEGIN and ----- END header and footer lines — without any truncation or whitespace errors.
- Navigated to GitHub repository Settings → Secrets and variables → Actions and deleted the existing invalid DEPLOY_KEY secret.
- Recreated the DEPLOY_KEY secret by pasting the correctly copied SSH private key and saved the updated secret.
- Pushed the code again to re-trigger the GitHub Actions pipeline.



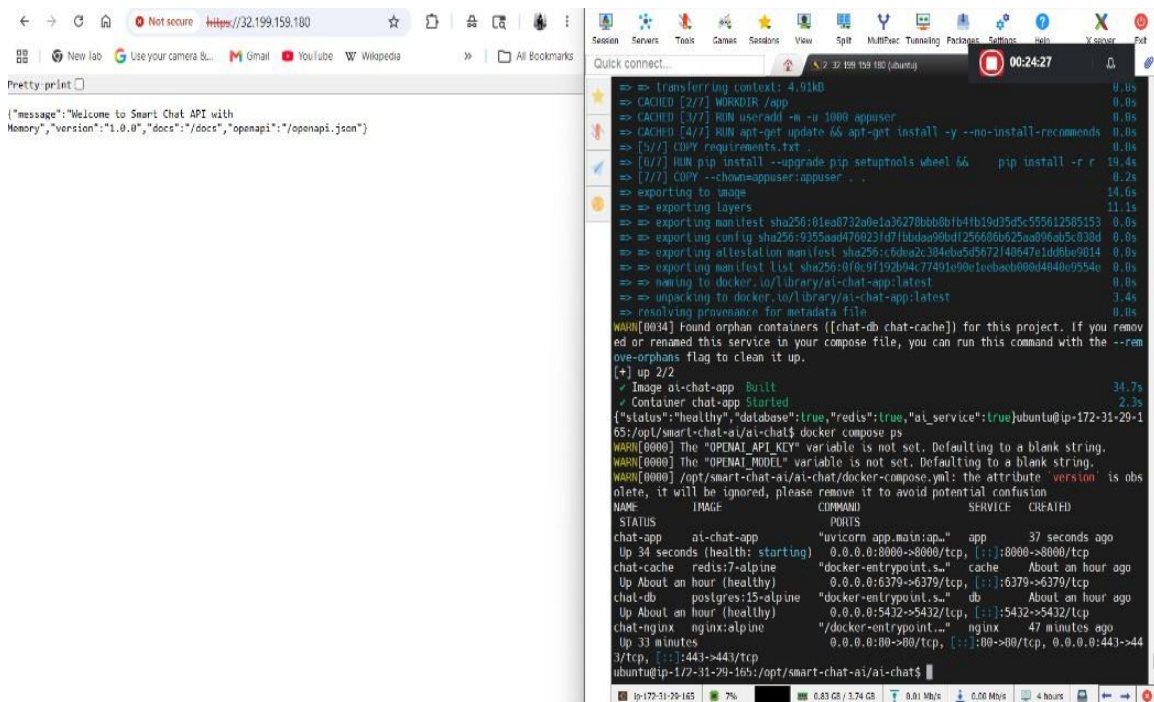
Push 3 — Full pipeline success

All three pipeline stages completed successfully. The application was built, tested, and deployed to the EC2 production server without any errors.

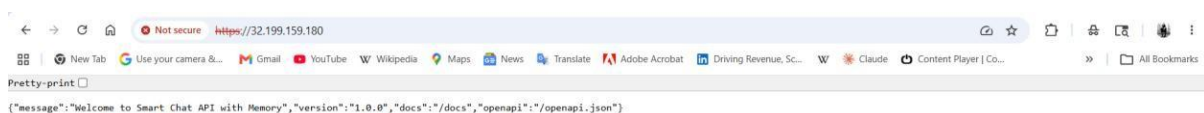
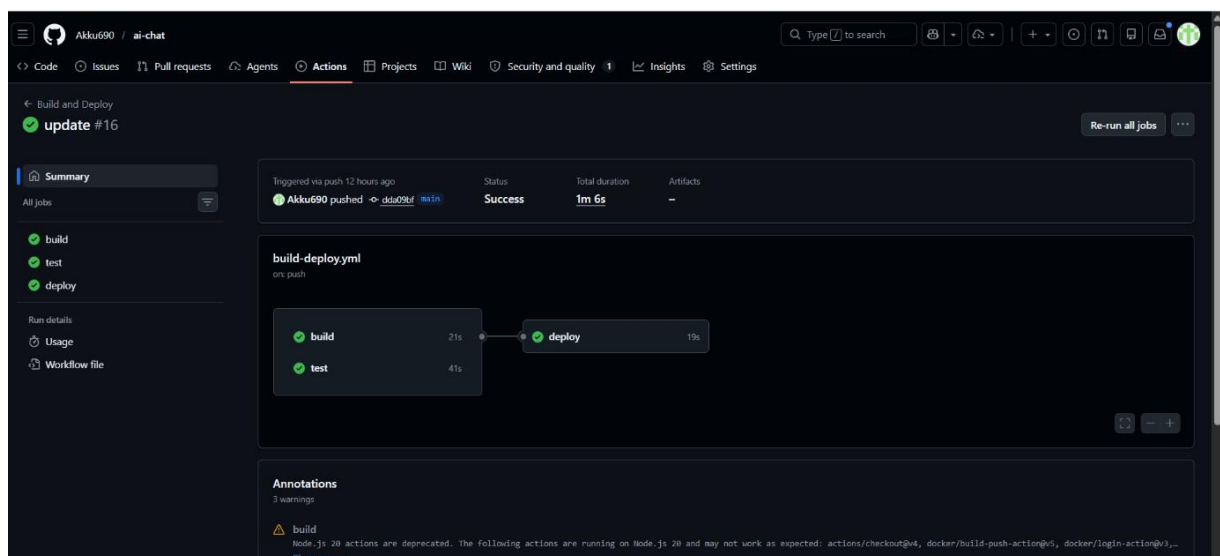


Error 6 : Next problem is at the time of health check deployment again get failed . But after that it resolved.

Application Health Verification



After resolving all the error our complete system get setup and working properly



Live Application Response

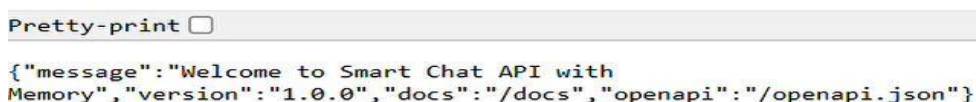


Fig 19 — API root endpoint response (HTTP) confirming the application is live

